VPC Peering Project Report

Name: Harsh Mahawar

Batch: AWS Cloud Computing

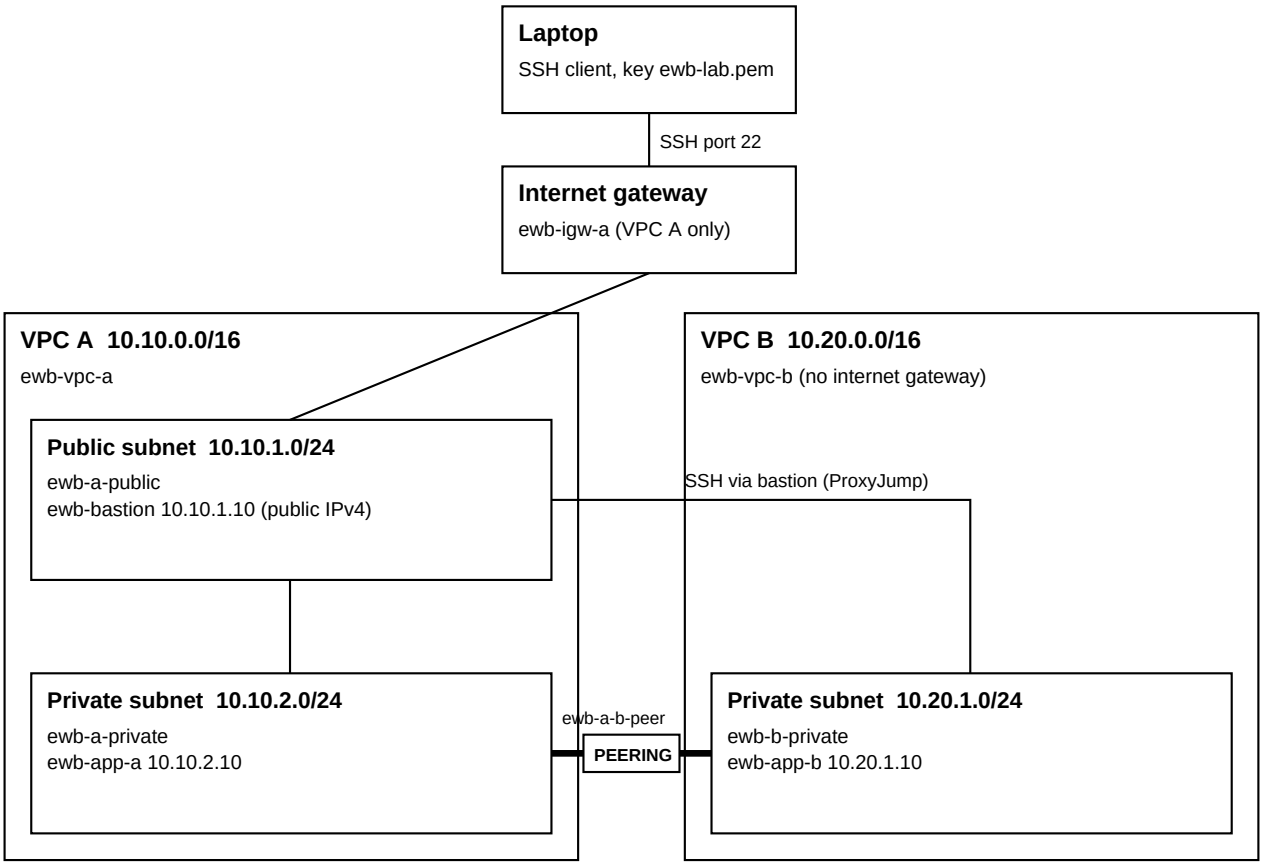
Project: VPC Peering (Project 01)

Region: Asia Pacific (Sydney), ap-southeast-2

Completion date: 21 October 2026

1. Architecture and design

This project connects two private VPCs with a VPC peering connection so that App-A (VPC A) and App-B (VPC B) can exchange traffic over their private IPv4 addresses. One public bastion host in VPC A is the only internet entry point and is used for administration with SSH ProxyJump. VPC B has no internet gateway.



Administration path: laptop to bastion to App-A or App-B (SSH ProxyJump). Project traffic: App-A to App-B over private IPs and the peering connection. Peering does not give VPC B internet access through VPC A.

Why these CIDRs

10.10.0.0/16 and 10.20.0.0/16 do not overlap, which is required for peering. Each /24 subnet sits inside its own VPC range.

Address plan and resource inventory

Resource / Name tag	Location	CIDR / private IP	AWS ID
ewb-vpc-a	VPC A	10.10.0.0/16	vpc-0bc7eecefd6ab67bd
ewb-a-public	VPC A subnet	10.10.1.0/24	subnet-0d9b83778a0f82afb
ewb-a-private	VPC A subnet	10.10.2.0/24	subnet-0718e16cc32fcbd37
ewb-vpc-b	VPC B	10.20.0.0/16	vpc-05c7086c26ae96e79
ewb-b-private	VPC B subnet	10.20.1.0/24	subnet-072aabea62fdd7c56
ewb-igw-a	Attached to VPC A	n/a	igw-0d96d913c9689535e
ewb-rt-a-public	VPC A	n/a	rtb-0dece2912b16199d5
ewb-rt-a-private	VPC A	n/a	rtb-01d8c374960ebabe7
ewb-rt-b-private	VPC B	n/a	rtb-0de0974900e185e2e
ewb-sg-bastion	VPC A	n/a	sg-0017b7a0ba98eae48

Resource / Name tag	Location	CIDR / private IP	AWS ID
ewb-sg-app-a	VPC A	n/a	sg-07cd59e0d3e0e2f1b
ewb-sg-app-b	VPC B	n/a	sg-04fedad89ec608747
ewb-bastion	ewb-a-public	10.10.1.10, public 54.206.117.147	i-071c0e686666e7709
ewb-app-a	ewb-a-private	10.10.2.10 (no public IP)	i-032e3541af08cd97c
ewb-app-b	ewb-b-private	10.20.1.10 (no public IP)	i-01046ce2475e38344
ewb-a-b-peer	VPC A to VPC B	n/a	pcx-0b5eafffe599bedc6
ewb-lab	EC2 key pair (RSA)	n/a	key name only

All resources are tagged Project = EWB-VPC-Peering and Student = Harsh Mahawar.

2. Subnets, internet gateway and route tables

Each subnet has an explicit route table association. The internet gateway is attached to VPC A only. The private subnets have no internet route and no public IPv4 address.

Route table: ewb-rt-a-public (associated with ewb-a-public)

Destination	Target	Purpose
10.10.0.0/16	local	Bastion to App-A
0.0.0.0/0	ewb-igw-a	Bastion internet access
10.20.0.0/16	pcx (ewb-a-b-peer)	Bastion SSH to App-B through ProxyJump

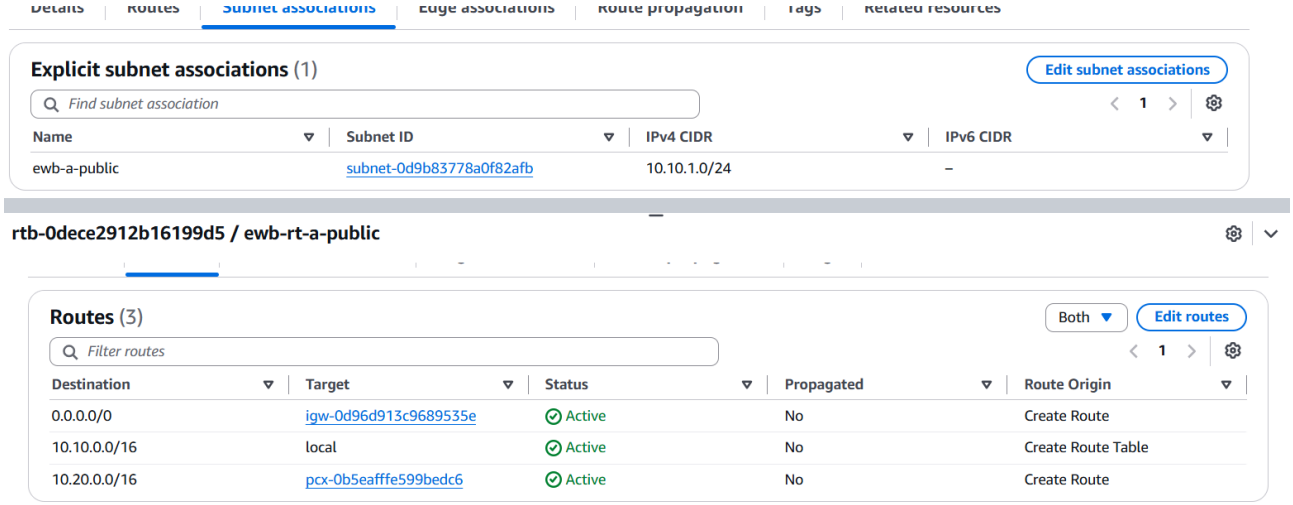
Route table: ewb-rt-a-private (associated with ewb-a-private)

Destination	Target	Purpose
10.10.0.0/16	local	Existing VPC A route
10.20.0.0/16	pcx (ewb-a-b-peer)	App-A to VPC B

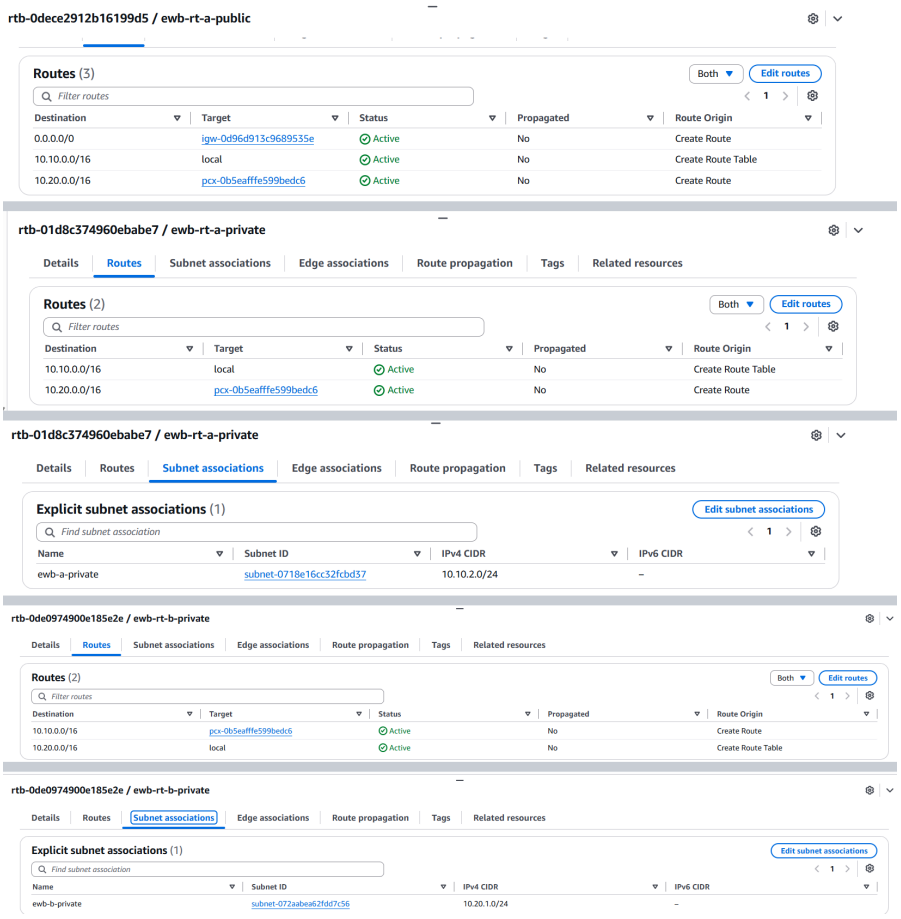
Route table: ewb-rt-b-private (associated with ewb-b-private)

Destination	Target	Purpose
10.20.0.0/16	local	Existing VPC B route
10.10.0.0/16	pcx (ewb-a-b-peer)	Return path to VPC A

Evidence 1: public subnet association and the public route table with its default route to the internet gateway



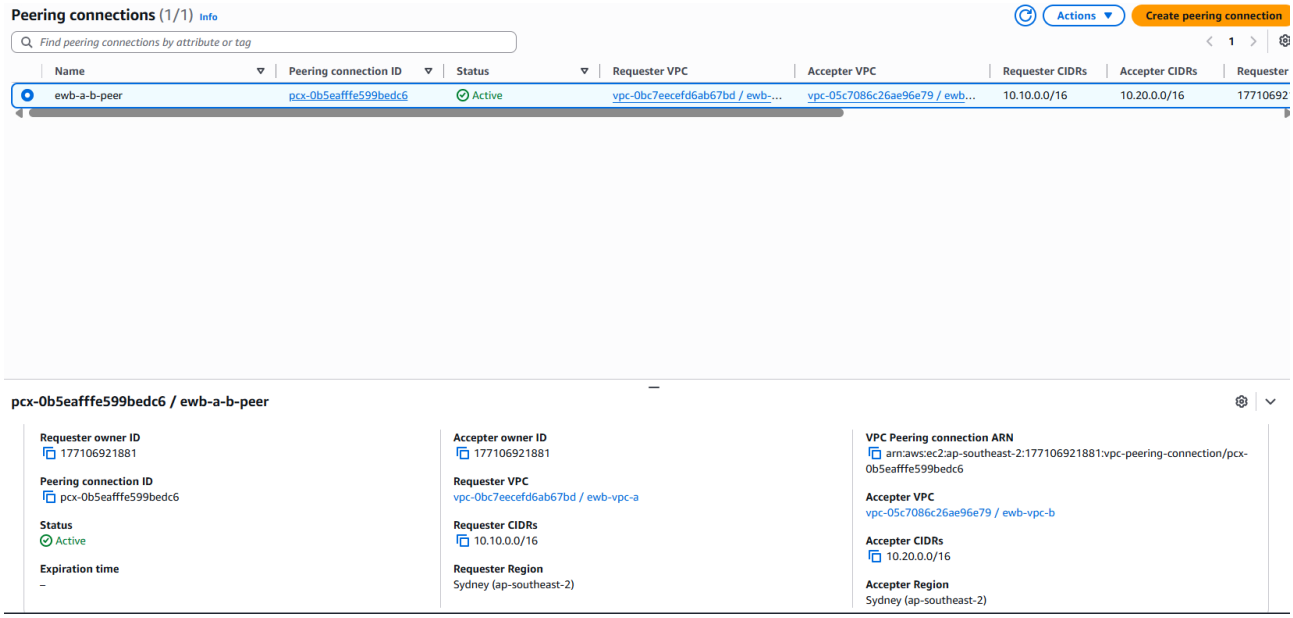
Evidence 2: all three route tables with the peering routes and subnet associations



3. Peering connection and security groups

Field	Value
Name	ewb-a-b-peer
Requester VPC	ewb-vpc-a (10.10.0.0/16), ID vpc-0bc7eecefd6ab67bd
Accepter VPC	ewb-vpc-b (10.20.0.0/16), ID vpc-05c7086c26ae96e79
Account / Region	My account / ap-southeast-2
Peering ID / status	pcx-0b5eafffe599bedc6 / Active (request accepted manually)

Evidence 3: peering connection ewb-a-b-peer, Active, with both VPC IDs

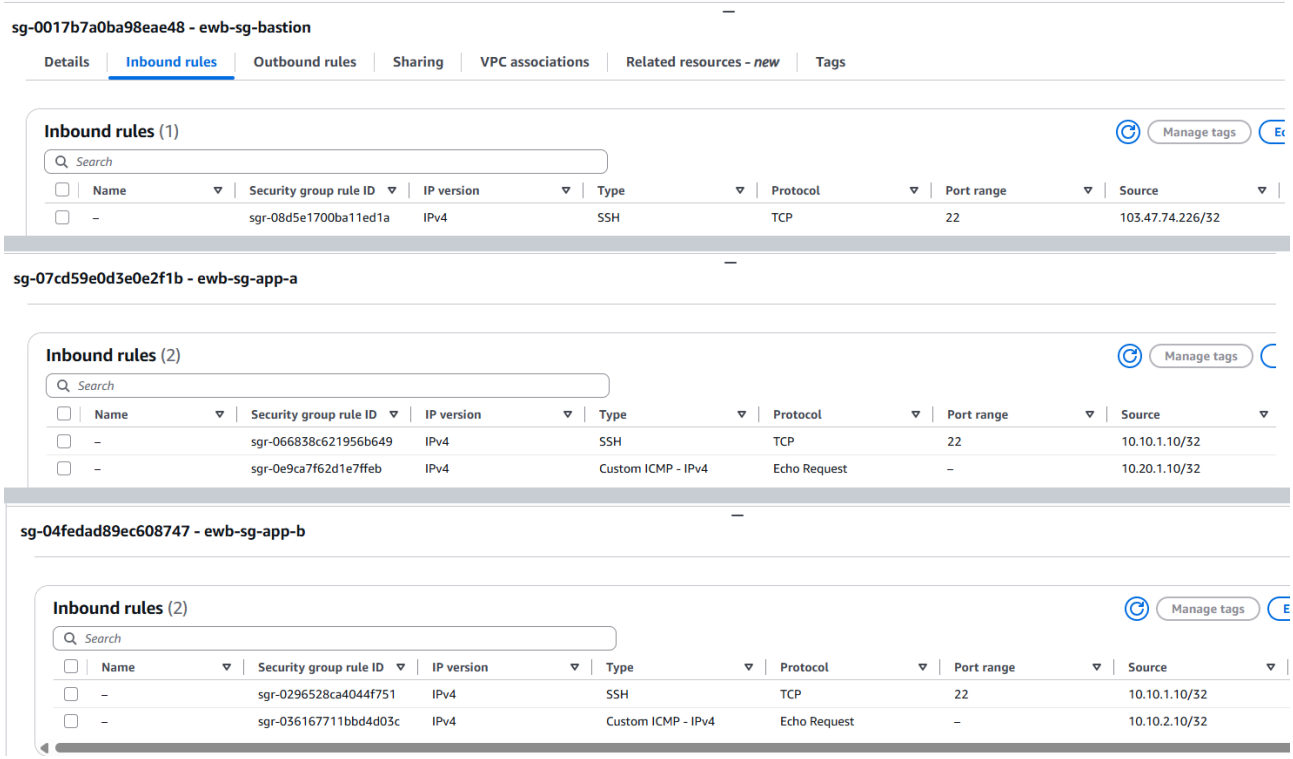


Inbound security group rules

Security group	Type / port	Source	Reason
ewb-sg-bastion (VPC A)	SSH / TCP 22	My laptop public IPv4 /32	Only my laptop can reach the bastion
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion)	Admin access only through bastion
ewb-sg-app-a (VPC A)	ICMP Echo request	10.20.1.10/32 (App-B)	Allow ping from App-B
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion)	Bastion private IP is the source seen by App-B
ewb-sg-app-b (VPC B)	ICMP Echo request	10.10.2.10/32 (App-A)	Allow ping from App-A

Outbound rules were left as the default allow-all. Security groups are stateful, so replies are allowed automatically. Network ACLs were left at the default.

Evidence 4: inbound rules of ewb-sg-bastion, ewb-sg-app-a and ewb-sg-app-b



sg-0017b7a0ba98eae48 - ewb-sg-bastion

Details | Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - *new* | Tags

Inbound rules (1)

Search

Manage tags

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
–	sgr-08d5e1700ba11ed1a	IPv4	SSH	TCP	22	103.47.74.226/32

sg-07cd59e0d3e0e2f1b - ewb-sg-app-a

Inbound rules (2)

Search

Manage tags

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
–	sgr-066838c621956b649	IPv4	SSH	TCP	22	10.10.1.10/32
–	sgr-0e9ca7f62d1e7ffeb	IPv4	Custom ICMP - IPv4	Echo Request	–	10.20.1.10/32

sg-04fedad89ec608747 - ewb-sg-app-b

Inbound rules (2)

Search

Manage tags

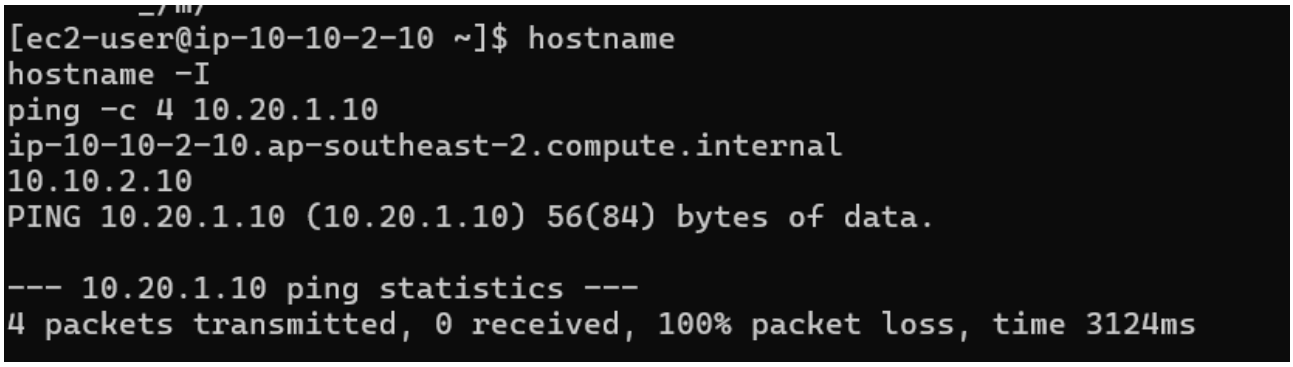
Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
–	sgr-0296528ca4044f751	IPv4	SSH	TCP	22	10.10.1.10/32
–	sgr-036167711bbd4d03c	IPv4	Custom ICMP - IPv4	Echo Request	–	10.10.2.10/32

4. Test evidence

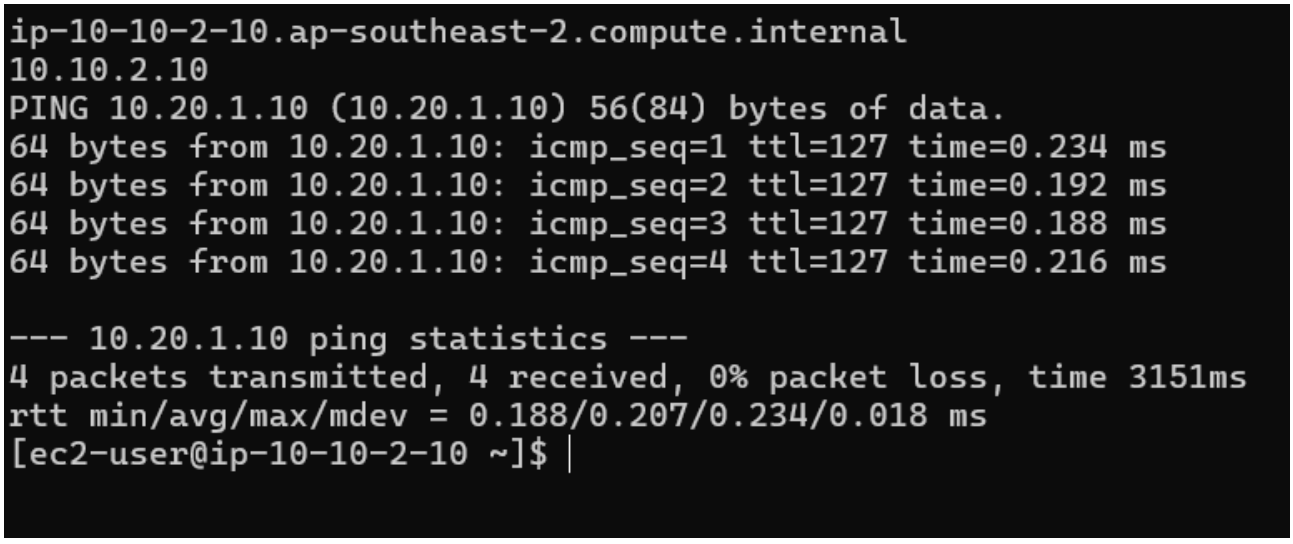
Every test uses private destination IPs. The output below is from my own run.

Test	Where / command	Expected result	Result
01 Baseline	App-A, before peering: ping -c 4 10.20.1.10	No replies	100% packet loss
02 Forward path	App-A, after routes: ping -c 4 10.20.1.10	4 replies, 0% loss	4 replies, 0% loss
03 Reverse path	App-B, after routes: ping -c 4 10.10.2.10	4 replies, 0% loss	4 replies, 0% loss
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B through bastion	Shell on App-B through bastion
05 Isolation	EC2 console, App-A and App-B	No public IPv4 on either host	No public IPv4

Evidence 5: baseline: App-A ping to 10.20.1.10 before peering (fails)



Evidence 6: forward path: App-A ping to 10.20.1.10 after peering (succeeds)



Evidence 7: reverse path: App-B ping to 10.10.2.10

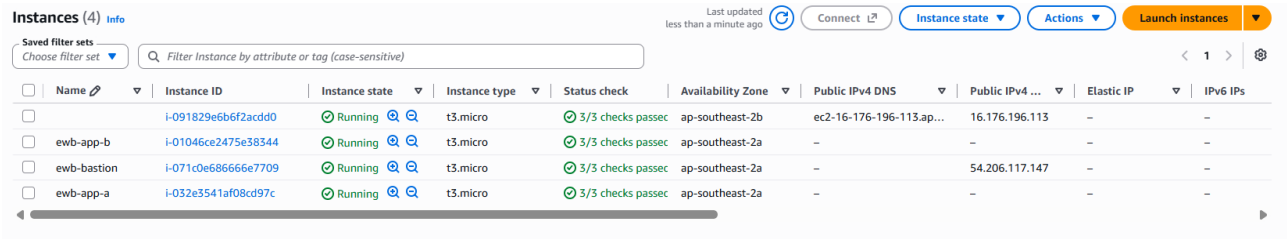
```
ip-10-20-1-10.ap-southeast-2.compute.internal
10.20.1.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.200 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=25.4 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.510 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.199 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3142ms
rtt min/avg/max/mdev = 0.199/6.588/25.445/10.887 ms
[ec2-user@ip-10-20-1-10 ~]$
```

Evidence 8: SSH from the laptop to App-B through the bastion

```
PS C:\Users\Harsh Mahawar> ssh ewb-app-b
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Oct 19 08:35:36 2026 from 10.10.1.10
[ec2-user@ip-10-20-1-10 ~]$
```

Evidence 9: EC2 instances: App-A and App-B have no public IPv4 address



5. Controlled failure and recovery

I removed only the inbound ICMP Echo request rule (source 10.10.2.10/32) from ewb-sg-app-b and repeated the forward ping from App-A. Then I restored the exact same rule and repeated the test. The SSH rule was left in place.

Evidence 10: rule removed: ping from App-A to 10.20.1.10 shows 100% packet loss

```
ip-10-10-2-10.ap-southeast-2.compute.internal
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3097ms

[ec2-user@ip-10-10-2-10 ~]$
```

Evidence 11: rule restored: ping from App-A to 10.20.1.10 shows replies again

```
ip-10-10-2-10.ap-southeast-2.compute.internal
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.195 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.172 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.170 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.173 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3152ms
rtt min/avg/max/mdev = 0.170/0.177/0.195/0.010 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Explanation

The routes and the peering connection stayed unchanged during this test, so the network path existed the whole time. Removing the ICMP rule made the destination security group drop the echo requests. This shows that routing and security groups are separate layers and both must allow the traffic. Restoring the rule restored the replies.

6. Forward path and return path

Forward path (App-A to App-B): App-A sends a packet to 10.20.1.10. The ewb-rt-a-private table has no local match for 10.20.0.0/16 but has a route for it to the peering connection, so the packet crosses into VPC B. ewb-sg-app-b then checks its inbound rule and accepts ICMP echo from 10.10.2.10/32.

Return path (App-B to App-A): App-B's reply is addressed to 10.10.2.10. The ewb-rt-b-private table has a route for 10.10.0.0/16 to the peering connection, so the reply crosses back. Without this route the request would arrive but the reply would be dropped. A one-way route is not enough. Security groups are stateful, so the reply is allowed without a separate inbound rule on App-A.

Bastion to App-B: With ProxyJump the bastion opens its own SSH connection to App-B. The bastion's subnet therefore needs the 10.20.0.0/16 peering route. The more specific route wins over the 0.0.0.0/0 internet route, and App-B sees the bastion's private IP (10.10.1.10) as the SSH source.

7. Cleanup

Done after saving all evidence, in this order:

- Terminated ewb-bastion, ewb-app-a and ewb-app-b. Deleted the ewb-lab key pair and the local .pem file.

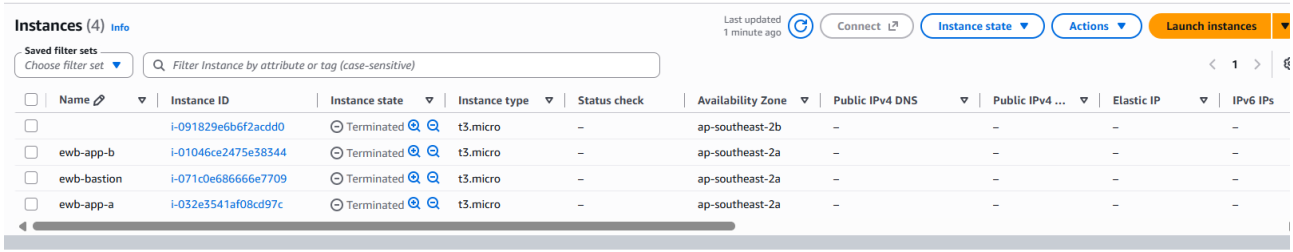
- Also terminated one untagged instance (i-091829e6b6f2acdd0, t3.micro, ap-southeast-2b, default VPC, key pair ewb-lab) that this project did not create. CloudTrail shows it was launched on 18 October 2026 at 21:40 UTC from my own browser session (my laptop public IP). I terminated it on 19 October 2026 at 19:48 UTC at my own request, without first confirming its owner or getting mentor approval. This should have been raised with my mentor or the account administrator before termination. Mentor informed: Yes.

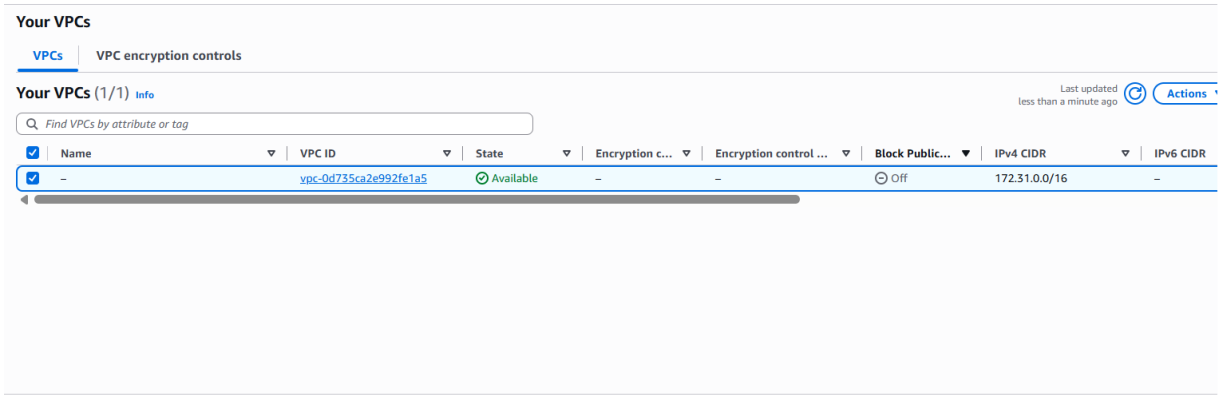
- Deleted the peering routes and ewb-a-b-peer, the three security groups, the three subnet associations, the three subnets, the three custom route tables, the internet gateway (detached first) and both VPCs.

- Checked the Sydney Region afterwards: no EWB VPCs, no running or stopped instances, no Elastic IPs and no volumes remain. The default VPC was not touched.

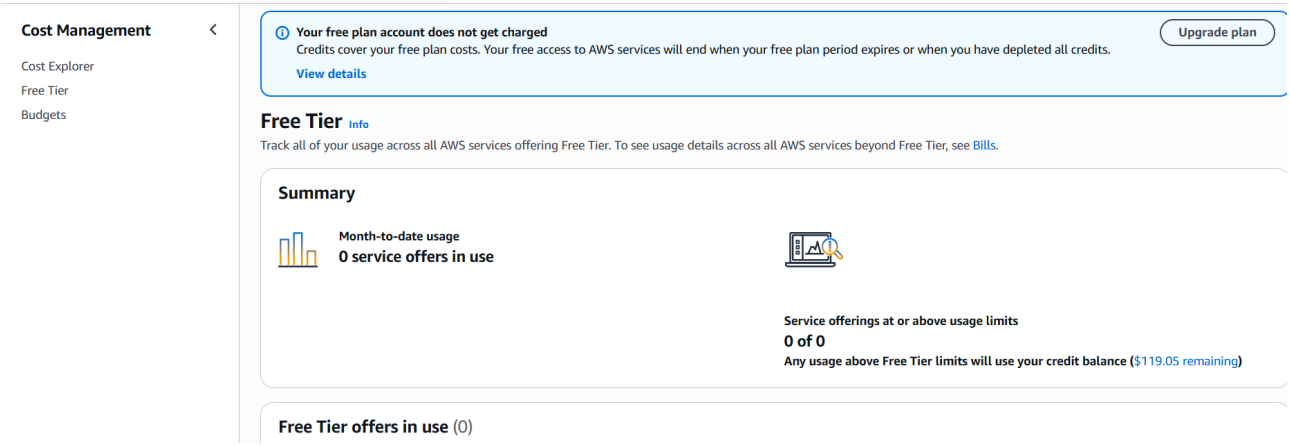
- Billing check after cleanup: the Free Tier page shows 0 service offers in use this month and $119.05 of credit remaining. The page banner reads "Your free plan account does not get charged."

Evidence 12: cleanup: instances terminated and no EWB VPCs remain





Evidence 13: billing: Free Tier page showing 0 service offers in use and $119.05 credit remaining



8. What I learned

Launching and connecting EC2 instances. I launched Amazon Linux 2023 instances and placed each one in a specific subnet with a fixed private IP. The bastion is the only instance with a public IP, and I connected to the private instances through it with SSH ProxyJump. This avoids giving the application servers public addresses.

Building a VPC. A VPC is defined by its CIDR block (here 10.10.0.0/16 and 10.20.0.0/16), then divided into subnets. A subnet becomes public only when its route table sends 0.0.0.0/0 to an internet gateway. The private subnets have no such route, so they cannot reach the internet directly.

Connecting two VPCs with peering. A peering connection alone does not carry traffic. Both route tables need a route to the other VPC, because the reply travels back on the return route. Before peering, the ping from App-A to App-B failed. After I added the routes, it succeeded with 0% loss.

Security groups and routing are separate layers. Removing only the ICMP rule on App-B made the ping fail while the routes stayed the same. Security groups are stateful, so replies to allowed traffic do not need their own inbound rule.

Cost, cleanup and ownership. Running instances and VPC resources keep billing until they are removed. Before terminating any resource, I should confirm who created it and get approval, because an untagged resource may belong to someone else. I terminated an untagged instance without that check. Tagging every resource makes ownership and cleanup clear.

Linux and terminal commands used in this project

Command	What it does in this project
ssh ewb-app-a	Connects to App-A using the host entry in the SSH config (through the bastion)
exit	Closes the current SSH session and returns to the laptop
hostname	Shows the instance name, for example ip-10-10-2-10 (proof of which host you are on)
hostname -I	Shows the instance's private IP address (10.10.2.10 for App-A)
ping -c 4 10.20.1.10	Sends 4 ICMP echo requests and reports replies and packet loss
clear	Clears the terminal screen so screenshots show only the current test
chmod 400 ewb-lab.pem	Makes the key file readable only by its owner (required by SSH on Linux and WSL)
mkdir -p ~/.ssh	Creates the .ssh folder (PowerShell: mkdir $HOME\.ssh -Force)
nano ~/.ssh/config	Edits the SSH config that defines the bastion and ProxyJump hosts
Move-Item / icacls	Windows PowerShell equivalents for moving the key and restricting its permissions

Other concepts covered: CIDR planning and non-overlapping ranges, route table association and the local route, the internet gateway, ICMP versus TCP port 22, the difference between a public IPv4 address and a private IP, and the role of the bastion as an administration entry point.

9. Conclusion

This project connected two private VPCs, VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16), with a VPC peering connection. App-A and App-B now communicate over their private IP addresses without public exposure. Administrative access is restricted: only the bastion has a public IP, only my laptop can reach the bastion, and the private instances accept SSH only from the bastion.

The tests confirmed the design. The baseline ping failed before peering. After peering and the routes were added, the ping worked in both directions with 0% loss, and SSH to App-B worked through the bastion. Removing the ICMP rule stopped the ping, and restoring it brought the ping back.

The main lesson is that connectivity needs both a route and a permission, in both directions. The peering route lets packets leave VPC A and the return route lets replies come back. Security group rules then decide what is allowed. A missing route or a missing rule on either side breaks the connection, which is why each test was needed to confirm the design works.

All resources created for the project were removed afterwards, and the cleanup was checked in the Sydney Region. The one untagged instance that was not created by the project is described in section 7.